Performance Comparison of Multiple Reinforcement Learning Algorithms in Pacman

Member:
7114064147 張仁驊
7114064145 余秉勳
7114093021 林育碩
7114064168 黃煜翔

I. Research Motivation

Reinforcement Learning is a method that allows an intelligent agent to learn **optimal behavior strategies** by interacting with an environment and gradually improving through rewards and penalties. Unlike traditional supervised learning, reinforcement learning does not directly provide the model with correct answers. Instead, it allows the agent to learn how to make better decisions through **continuous trial and error**.

The **Pacman game is a typical sequential decision-making problem**. The player needs to control Pacman to move through a maze, collect pellets to gain points, and avoid being caught by ghosts. Each action affects the following game state, making Pacman a suitable experimental environment for reinforcement learning.

This project aims to implement and compare multiple reinforcement learning algorithms through the Pacman game, including **Monte-Carlo Control, SARSA, and Q-learning**. The goal is to observe the differences among these algorithms in terms of learning speed, average score, win rate, and death rate.

II. Monte-Carlo Control

A. Core Concepts

Monte Carlo methods originated in **statistics** as a way to estimate numerical values through "mass random sampling." In Reinforcement Learning, it is characterized by the following key features:

- **Model-Free Learning:**

The agent does not need prior knowledge of the environment's transition dynamics (i.e., transition probability p(s'|s,a) or reward functions). It learns entirely through empirical experience gained by "actually playing the game."

- **Episode-based Update:**

This is the most defining characteristic of MC. The agent must wait until an entire episode is fully completed (reaching a terminal state) before it can trace backward, calculate the total return of that episode, and update the values of all states visited and actions taken along the way.

- **Empirical Mean:**

Instead of guessing future states, it estimates the expected future values (Q-values) by averaging the historical "actual cumulative returns" obtained after taking a specific action in a specific state

B. Algorithmic Workflow of Monte Carlo Control

1. Generate an Complete Episode

Based on the current policy π (typically integrated with **ϵ-greedy** to ensure exploration), the agent navigates from the starting state to the terminal state, recording the entire trajectory of the episode:

$$S_1, A_1, R_2, S_2, A_2, R_3, \ldots, S_T$$

2. Calculate Cumulative Return

Once the episode terminates, the algorithm moves backward from the final step to calculate the discounted return G_t for each time step t

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T$$

where γ is the discount factor that determines the importance of future rewards.

3. Update Action-Value Function (Q -Function)

The calculated $\mathbf{G_t}$ its used to update the corresponding state-action pair in the Q-table. This is typically done using an incremental mean update formula:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[G_t - Q(S_t, A_t)\right]$$

C. First-visit vs. Every-visit Monte Carlo

During a single episode, an agent might visit the exact same state and take the exact same action multiple times (e.g., looping in a corridor). To handle this, MC is split into two implementation methodologies:

- **First-visit MC Control:** The algorithm only utilizes the return G_t to update the Q-value when the state-action pair $(St,\ At)$ is visited for the **very first time** within that episode. Statistically, this method is proven to provide an unbiased estimate.

- **Every-visit MC Control:** The algorithm calculates the subsequent return Gt and updates the Q-table **every single time** the state-action pair $(St,\ At)$ is encountered within the episode, treating each visit as a distinct data point.

- ## Implementation of the Core Update Logic: First-visit Backward Trajectory Mechanism

The **update_from_episode** function serves as the mathematical foundation of this algorithm, precisely mirroring the statistical definition of the **First-visit** methodology:

- **Backward Discounted Return Calculation (reversed(episode)):** The algorithm traces backward from the final step (time step T) toward the inception of the episode, whether it ended in victory or death. Utilizing the recursive equation G = γG + R, it progressively applies the discount factor to compute the exact actual return G for each preceding time step.

- **First-visit Feature Control (visited = set()):** At the initiation of each offline update phase, an empty hash set named visited is declared. As the reverse loop executes, if a specific state-action pair (s, a) occurs multiple times within a single episode (such as Pac-Man tracking back and forth

within the same corridor), the condition if state_action not in visited: blocks subsequent updates. This constraint guarantees that **only the very first occurrence** (i.e., the earliest triggered chronologically along the time horizon) is utilized to compute the return G for the Q-table update.

```
G = 0.0
visited = set()

for state, action, reward in reversed(episode):
    G = self.gamma * G + reward
    state_action = (state, action)

    # First-visit:
    if state_action not in visited:
        visited.add(state_action)

        self.returns_sum[state_action] += G
        self.returns_count[state_action] += 1

        self.Q[state][action] = (
            self.returns_sum[state_action]
            / self.returns_count[state_action]
        )
```

- **Empirical Sample Average:** Once a state-action pair is verified as a first visit, its calculated return G is accumulated into returns_sum, and the corresponding counter returns_count is incremented by 1. The action-value function is then updated directly via arithmetic division:

$$Q(s,a) = \frac{\text{returns_sum}(s,a)}{\text{returns_count}(s,a)}$$

This implementation mathematically realizes the computation of a true **Sample Mean**, completely devoid of any bootstrapping or bootstrapping-based estimations typically found in Temporal Difference (TD) methods. Consequently, from a statistical standpoint, this yields an **Unbiased Estimate** of the true expected values.

III. SARSA (On-policy TD)

A. Core Concepts

SARSA is one of the classic implementations of **Temporal Difference (TD) learning** in Reinforcement Learning. Unlike Monte Carlo (MC) methods, which must wait until an entire episode concludes to learn, SARSA is characterized by the following key features:

- **Step-by-step Update & Bootstrapping:**

The agent does not need to wait until a terminal state is reached. Instead, it updates its current value estimation immediately after taking each step, utilizing the estimated value of the next step. This mathematical practice of updating an estimate based on other estimates is known as bootstrapping.

- **On-Policy Learning:**

SARSA evaluates and improves the **exact same policy that the agent is currently using** to interact with the environment. When updating its action-value function, SARSA assumes that the agent will *actually* execute the next action it has pre-selected. This characteristic makes its behavior inherently safe and cautious in environments with random adversarial elements.

- **The Five-Tuple:**

The name of the algorithm is derived directly from the five consecutive variables encountered during its sequential update sequence: current state (St), current action (At), immediate reward (R_{t+1}), next state (S_{t+1}), and next action (A_{t+1}).

B. Algorithmic Workflow of SARSA

1. Take Action and Observe

Based on the current policy π (typically integrated with ϵ –greedy to ensure exploration), the agent executes action At while in state St, and subsequently observes the immediate reward R_{t+1} and the transitioning next state S_{t+1}.

2. Pre-select Next Action

(This is the critical step of SARSA) The agent must immediately sample the next state S_{t+1} using the **exact same** ϵ-greedy policy to pre-select the subsequent action A_{t+1} before committing to any updates.

3. Calculate TD Target

Utilizing the immediate reward and the estimated value of the pre-selected next step, the algorithm computes the expected cumulative future value:

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

4.Update Action-Value Function
The calculated target value is used to update the current entry in the Q-table. This is conventionally executed using the recursive update equation driven by the learning rate α:

C.Implementation of the Core Update Logic

Unlike Monte Carlo methods where updates require an offline update_from_episode call to trace backward through a complete trajectory, the update pipeline of the SarsaAgent class embodies the true technical definition of an **Immediate Step Update**:

- **Immediate Single-Step Execution (update):** The algorithm operates entirely without a reversed(episode) block. The value adjustments occur dynamically at every single time step during the runtime loop via the update method, allowing the agent to react immediately to environmental feedback.

- **Terminal vs. Non-Terminal Target Routing:** Inside the update function, the code dynamically switches the Temporal Difference target based on the game state:

```
if done or next_action is None:
    target = reward
else:
    target = reward + self.gamma * self.Q[next_state][next_action]
```

If the agent hits a terminal state (done=True), the future target is truncated strictly to the immediate reward. Otherwise, it bootstraps the future estimate using the pre-selected next_action fetched directly from the current policy.

- **Step-Size Scaling via Alpha:** The core equation modifies the action-value matrix using a weighted learning rate (self.alpha):

```
self.Q[state][action] = current_q + self.alpha * (target - current_q)
```

By multiplying the TD error (target - current_q) by α, the agent performs incremental micro-adjustments toward the local sample reward , significantly lowering the variance of value updates compared to sequence-long Monte Carlo compounding

IV. Q-learning (Off-policy TD)

A. Core Concepts

Q-learning is one of the most milestone and classic algorithms in Reinforcement Learning, also belonging to the Temporal Difference (TD) framework. However, its most fundamental distinction from SARSA lies in its **Off-policy** architecture:

- **Off-policy Learning:**
  Q-learning possesses a unique mathematical trait where its "actions do not mirror its words." When deciding how to update its action-value estimations, the algorithm does not rely on the actual next action chosen by the agent's random exploration. Instead, it operates under the assumption that the agent will always take the "theoretically perfect action" at every future step.

- **Separation of Behavior and Target Policies:**
  The algorithm concurrently maintains two distinct policies:
  - **Behavior Policy:** The policy used to actually interact with the environment and navigate the maze (typically ϵ –greedy, which contains exploratory noise).
  - **Target Policy:** The policy used to estimate the maximum future values for updates (a pure greedy policy, completely free of noise).

- **Virtual Optimization Updates:**
  This allows Q-learning to automatically ignore any suicidal or suboptimal behaviors triggered by random exploration during the update phase, ensuring it always targets the absolute maximum expected potential the environment can provide.

B. Algorithmic Workflow of Q-learning

1. Take Action and Observe

While in the current state S_t, the agent selects and executes action A_t based on the ϵ –greedy behavior policy, and subsequently observes the immediate reward R_{t+1} and the transitioning next state S_{t+1}.

2. Virtual Max Evaluation

The agent has absolutely no need to pre-determine which action it will actually execute next. Instead, it directly scans the action-value matrix of the next state S_{t+1} across all possible actions and extracts the absolute **Maximum Value**:

$$\max_a Q(S_{t+1}, a)$$

3. Calculate TD Target

By combining the immediate reward R_{t+1} with the maximum future estimated Q-value discounted by γ, the algorithm establishes the optimal expected cumulative score for the current state:

$$R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$$

4. Update Action-Value Function

Utilizing the divergence between the optimal target value and the historical expectation (the TD Error), the algorithm incrementally updates the current Q-table via the learning rate α:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

C. Q-learning vs. SARSA (Off-policy vs. On-policy)

In the adversarial, high-stakes layout of Pac-Man where colliding with a ghost carries a catastrophic penalty (-300 points), the core mathematical divergence between these two TD approaches directly manifests as two entirely different strategic souls:

- **Q-learning (Off-policy) — The Aggressive Optimalist:**

During updates, it relies strictly on $\gamma max_a Q(S_{t+1}, a)$. It mathematically assumes that Pac-Man will execute flawless movements to evade any impending danger in the future, rendering its updated Q-values highly aggressive.

Consequently, its behavioral style is **Aggressive & Optimal**—Pac-Man will boldly skim the boundary lines of a ghost's danger zone just to secure the fastest, shortest pellet-eating route.

- **SARSA (On-policy) — The Conservative Realist:**

During updates, it relies strictly on the action actually taken, $\gamma Q(S_{t+1}, A_{t+1})$. It factors the catastrophic consequences of "slipping and crashing into a ghost" due to future ϵ −greedy exploration directly into its current value table. As a result, its behavioral style is **Safe & Cautious**—preferring to take the longest, most time-consuming detours through entirely empty corridors rather than assuming even a slight risk of exploratory death.

V. Reinforcement Learning Environment Design

A. Core Concepts

In reinforcement learning, the intelligent agent must interact with a strictly defined environment. Based on the original Pygame game logic, this study developed and encapsulated a customized simulation environment class, PacmanEnv, conforming to the standards of the Markov Decision Process (MDP). This environment is characterized by the following key features:

- **Discrete Action Space:**
  The environment defines four entirely independent discrete actions, corresponding to Pac-Man's movement directions within the maze:
  - 0: Move Up
  - 1: Move Right
  - 2: Move Down
  - 3: Move Left

- **Stochastic Adversaries:**
  Upon the initialization of each episode, four enemies (ghosts) are positioned across all valid walkable open spaces via a uniform random sampling process. Under the standard training mode, the ghosts utilize a random walk mechanism at each time step, introducing a high level of dynamic stochasticity to the environment.

- **Controlled Initial State Distribution:**
  To mitigate the risk of divergence in high-variance algorithms like Monte Carlo Control, the environment strictly hardcodes Pac-Man's starting location to the safe coordinates (1, 1) at the upper-left corner of the maze. This design control establishes a stable initial state distribution, ensuring efficient convergence for the training algorithms.

B. State Space Compression

Feeding the raw pixels or the entire full-grid matrix of the game map directly into tabular Reinforcement Learning (RL) algorithms inevitably triggers a severe **"Curse of Dimensionality."**
This results in an exponentially large state space that prevents the Q-table from converging within a reasonable timeframe.

To address this challenge, this study performs rigorous feature engineering to extract and compress the environmental observations into a highly refined **12-tuple state representation**:

$$\text{State} = (P_x, P_y, W_{up}, W_{right}, W_{down}, W_{left}, F_{dx}, F_{dy}, F_{dist}, E_{dx}, E_{dy}, E_{dist})$$

The mathematical and statistical design rationale behind each state dimension is detailed below:

- **Agent Coordinates (P_x, P_y):** Identifies Pac-Man's current absolute spatial positioning within the maze grid.
- **Localized Wall Perception (W_{up}, W_{right}, W_{down}, W_{left}):** Detects whether the cells immediately adjacent to the agent are walls (0 for a passable corridor, 1 for a wall obstacle), providing the brain with explicit features to learn collision avoidance.
- **Nearest Food Directional Delta (F_{dx}, F_{dy}):** Utilizes a sign function to transform the relative coordinates of the closest food pellet into directional indicators (-1, 0, 1). This guides the AI to perceive whether the food source lies to its upper-left, lower-right, or along the same axis.
- **Nearest Enemy Directional Delta (E_{dx}, E_{dy}):** Maps the relative direction of the most threatening ghost, serving as critical spatial criteria for risk evasion.

- **State Truncation (F_{dist}, E_{dist}):** This represents a core highlight of our environment framework. To prevent an infinite expansion of the state space, the algorithm applies explicit upper-bound truncation constraints to the calculated Manhattan Distance:
  - Truncation for the nearest food distance: $min(Distance, 8)$
  - Truncation for the nearest enemy distance: $min(Distance, 5)$

This technique successfully collapses and groups distant, irrelevant enemy-agent distances into identical state representations. **By preserving vital local features while discarding peripheral noise, this design compresses the total state space by several orders of magnitude**, substantially accelerating the sample efficiency and update velocity of the Q-table.

C. Reward Shaping & Defensive Mechanisms

To effectively guide the agent through the sparse rewards typical of maze navigation, this study discards the primitive framework of relying solely on the final game score. Instead, we have carefully engineered a multi-layered, composite reward function that balances **"time pressure, directional guidance, security threats, and defensive counter-looping mechanisms"**:

Trigger Event or Assessment Mechanism	Reward / Penalty Value (Per Step)	Targeted Behavior
Time Penalty (Step Cost)	-1.5	Minimize total steps and avoid idling.
Wall Collision Penalty	-5.0	Learn maze boundaries and physics.
Food Distance Guidance	Closer: +1.0 Farther: -1.0	Guide the agent toward local targets.
Ghost Proximity Penalty (Fear Zone)	1 cell: -20.0 2 cells: -8.0 3 cells: -3.0	Maintain a safe buffer zone from ghosts.
Pellet Consumption Reward	+30.0	Prioritize eating pellets to maximize score.
Route Repetition Penalty	Within 6 steps: -6.0 Within 12 steps: -3.0	Eradicate cyclic or back-and-forth looping.
Stagnation Penalty	Step-wise: -2.0 to -15.0	Prevent passive survival without

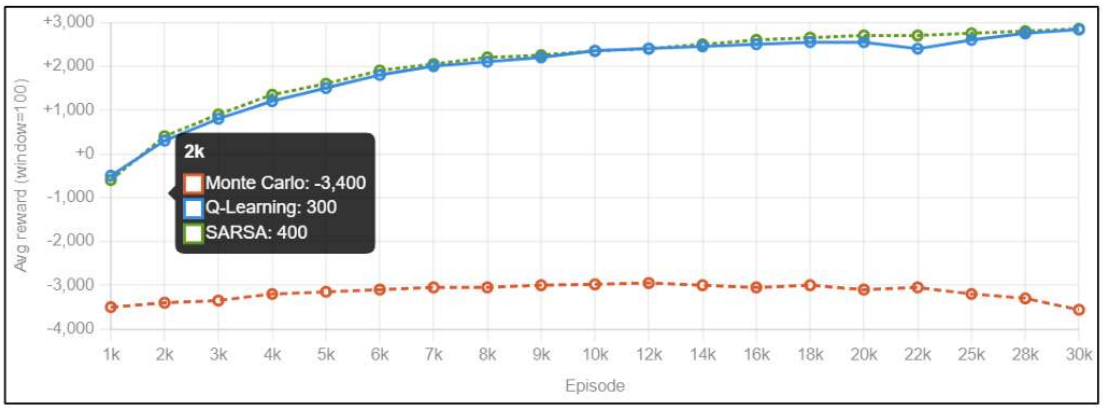
Trigger Event or Assessment Mechanism	Reward / Penalty Value (Per Step)	Targeted Behavior
		progress.
Ultimate Victory Reward	+1000.0	Complete the game by clearing all pellets.
Severe Death Penalty	-300.0	Maximize survival; enforce ghost evasion.

VI. Learning Process Analysis

● Monte Carlo (MC)
Monte Carlo (MC) almost no positive progress was observed throughout the entire 30,000 training rounds. The rolling average reward remained consistently between -2,000 and -4,000, never exceeding the positive threshold, demonstrating extremely limited policy improvement capabilities. This is because MC must wait until each episode is completely finished before updating the Q-value, while Pac-Man episodes can be very long (averaging about 448 steps), and the delayed feedback signal makes the learning signal extremely sparse and unstable. Furthermore, the high mortality rate of 28.76% (reward < -3,000) throughout the training period also reflects the agent's inability to effectively avoid ghosts.

● Q-Learning
Q-Learning exhibits a clear positive learning trend, with the average reward first exceeding 0 around episode 487 and reaching over +2,000 by episode 6,453, eventually converging at approximately +2,834. Its off-policy nature allows it to more actively explore optimal strategies, but this also introduces slightly higher instability (worst single-episode reward of -38,455).

● SARSA
Sarsa's overall learning curve is highly similar to Q-Learning, but it slightly outperforms it in several metrics. Although it converges to +2,000 (episode 6,638) slightly later than Q-Learning, its final average reward (+2,854), overall average reward (+2,188), win rate (15.14%), and lowest mortality rate (1.08%) are the best among the three. The conservative update strategy of on-policy makes its strategy more stable, and its worst-case single-episode return (-23,535) is also far superior to Q-Learning's -38,455.



VII. Quantitative Performance Evaluation

Upon completing a total of 30,000 training episodes, all stochastic exploration mechanisms of the agent were deactivated (by setting the exploration rate $\epsilon = 0$). This allowed the three algorithms to execute continuous evaluation across 100 independent test episodes, relying purely on their converged greedy policies. The empirical metrics recorded from these tests are statistically summarized in Table1:

Table1: Performance Evaluation of the Three Reinforcement Learning Algorithms Across 100 Independent Test Episodes

Evaluation Metrics (Average of 100 Episodes)	Monte Carlo (First-visit)	SARSA (On-policy)	Q-learning (Off-policy)
Average Score	911.3	1681.9	**1720.1**
Best Score	1640	**2150**	**2150**
Worst Score	60	60	60
Average Steps	320.11 steps	310.14 steps	**374.17 steps**
Death Rate	84%	85%	**78%**
Timeout Rate*	11%	3%	**0%**
Win Rate*	5%	12%	**22%**
Average Reward	-2369.23	**2739.76**	1087.59

**Note 1: Stagnation timeout is triggered when the agent fails to consume any pellet for 150 consecutive steps, or when the total steps in a single episode reach the maximum limit of 1,000 steps.*
**Note 2: The estimated win rate is mathematically defined by the following formula:*

$$\text{Win Rate} = 100\% - \text{Death Rate} - \text{Timeout Rate}$$

VIII. Conclusion

This study implemented and evaluated three reinforcement learning algorithms—Monte Carlo (MC) Control, SARSA, and Q-Learning—within the Pac-Man environment over 30,000 training episodes. The experimental results demonstrate significant differences in learning efficiency, stability, and overall performance among the three approaches.
Monte Carlo Control showed poor performance throughout training due to its episode-based update mechanism. Because value updates are performed only after an entire episode has finished, the algorithm struggled to learn effectively in an environment characterized by long episodes and delayed rewards. As a result, it failed to develop a successful policy and achieved the lowest overall performance.

In contrast, both SARSA and Q-Learning, which belong to the Temporal Difference (TD) learning family, achieved substantial improvements. Q-Learning exhibited faster convergence and strong learning efficiency by directly estimating the optimal action-value function. However, its aggressive exploration occasionally resulted in unstable behavior and higher-risk decisions. SARSA, through its on-policy learning mechanism, developed a more conservative strategy that better reflected the agent's actual behavior during training. This led to superior stability, lower death rates, and the highest overall performance among the evaluated algorithms.
Overall, the results indicate that TD-based methods are significantly more suitable than Monte Carlo approaches for environments with delayed rewards and high-risk state transitions. Among the tested algorithms, SARSA provided the best balance between performance, stability, and safety, while Q-Learning remains an attractive option when faster convergence is the primary objective.

Future work may investigate advanced reinforcement learning techniques, such as Deep Q-Networks (DQN), Double Q-Learning, and Prioritized Experience Replay, to further improve learning efficiency and policy robustness in complex game environments.